

LAPORAN TUGAS
DEADLOCK SISTEM OPERASI



Dosen Pengampu :

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

MOH. Ayatullah Mutawakki. F {2410651011}

Tanggal: 10-November-2025

PROGRAM STUDY TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

Contents

DAFTAR ISI	2
ISI LAPORAN	1
A. SOAL PEMROGRAMAN – TUGAS 1: Analisis Deadlock	1
B. Tugas 2 — Implementasi Pencegahan Deadlock.....	6
C. Tugas 3: Program Banker's Algorithm.....	11
D. TUGAS 4: Kasus Nyata - Simulasi Deadlock Transfer Uang	16
KESIMPULAN	23

ISI LAPORAN

A. SOAL PEMROGRAMAN – TUGAS 1: Analisis Deadlock

1. Buat File

nano deadlock_simple.c

kode lengkap deadlock_simple.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA;
pthread_mutex_t lockB;

void* thread1(void* arg) {
    pthread_mutex_lock(&lockA);    // Thread 1 mengambil Lock A dulu
    printf("Thread 1: locked A\n");
    sleep(1);                      // Biar timing-nya pas

    pthread_mutex_lock(&lockB);    // Lalu mencoba Lock B
    printf("Thread 1: locked B\n");

    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void* thread2(void* arg) {
    pthread_mutex_lock(&lockB);    // Thread 2 mengambil Lock B dulu
    printf("Thread 2: locked B\n");
    sleep(1);

    pthread_mutex_lock(&lockA);    // Lalu mencoba Lock A
    printf("Thread 2: locked A\n");
```

```

pthread_mutex_unlock(&lockA);
pthread_mutex_unlock(&lockB);
return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

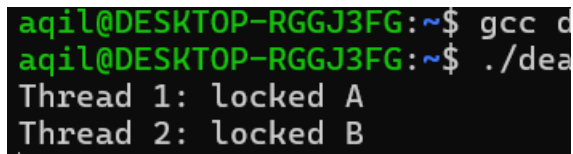
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lockA);
    pthread_mutex_destroy(&lockB);

    return 0;
}

```

2. Output



```

aqil@DESKTOP-RGGJ3FG:~$ gcc d
aqil@DESKTOP-RGGJ3FG:~$ ./dea
Thread 1: locked A
Thread 2: locked B

```

3. Analisis Masalah

a. Pemahaman Soal

Pada tugas ini, program `deadlock_simple.c` dijalankan untuk melihat fenomena deadlock pada dua thread yang menggunakan dua buah mutex. Program memiliki dua thread (T1 dan T2), yang masing-masing melakukan penguncian resource secara terbalik:

- Thread 1: lock A ke lock B
- Thread 2: lock B ke lock A

Polanya menyebabkan kedua thread saling menunggu selamanya yakni terjadi deadlock.

b. Megeidentifikasi Masalah Sinkronisasi

Masalah utama terjadi karena:

1. Setiap thread mengunci satu resource dan mencoba mengunci resource lain yang sedang digunakan thread lain.
2. Penguncian dilakukan dengan urutan yang berbeda, sehingga menyebabkan circular wait.
3. Tidak ada mekanisme timeout, try lock, ataupun lock ordering.

c. Desain Solusi (Analisis Sebelum Perbaikan)

Tujuan tugas adalah memahami kenapa deadlock terjadi, bukan memperbaikinya.

Namun secara konsep, deadlock bisa dicegah dengan:

- Lock Ordering
- Try-lock
- Timeout
- Resource hierarchy
- Avoidance (e.g., Banker's Algorithm)

Tugas ini fokus pada identifikasi masalah, bukan perbaikan.

4. Implentasi

a. Penjelasan Sintax

pthread_mutex_lock(&mutexA); sleep(1); pthread_mutex_lock(&mutexB);
pthread_mutex_lock(&mutexB); sleep(1); pthread_mutex_lock(&mutexA);

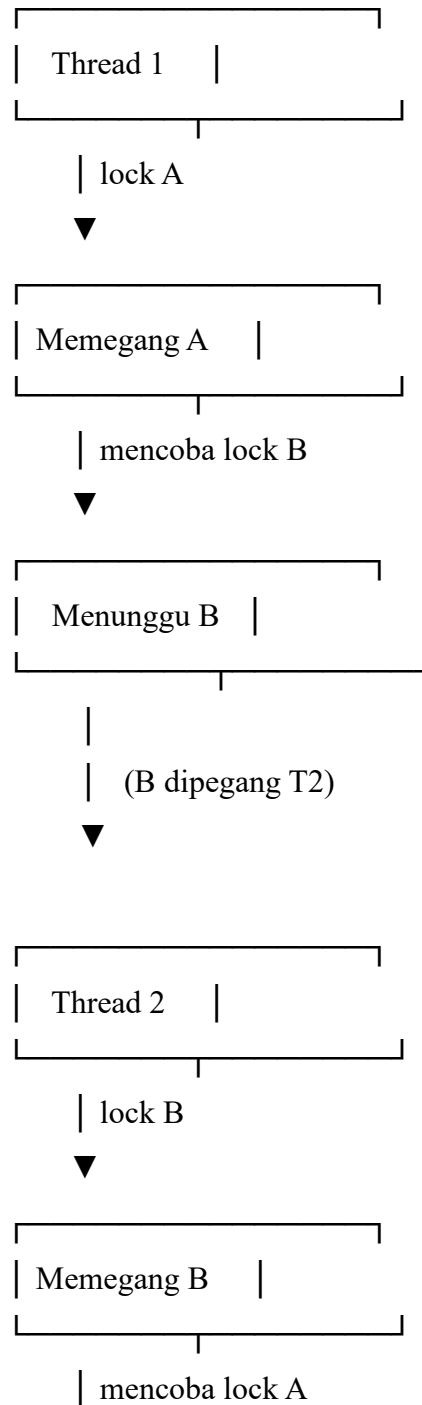
Penjelasan:

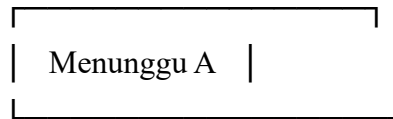
- Thread 1 mengunci mutexA, lalu menunggu mutexB.
- Thread 2 mengunci mutexB, lalu menunggu mutexA.
- Karena masing-masing sudah memegang satu kunci dan menunggu kunci yang lain, keduanya tidak bisa lanjut. Inilah penyebab deadlock.

b. Penejelasan Algoritma

- Dua thread dibuat.
- Thread 1 -- kunci mutex A -- tunggu -- kunci B.
- Thread 2 -- kunci mutex B -- tunggu -- kunci A.
- Karena kedua mutex saling dikunci oleh thread berbeda, dan masing-masing membutuhkan mutex lain, program berhenti selamanya.

c. Flowchart Deadlock Program





Siklus:

T1 menunggu B -- T2 menunggu A -- kembali ke T1.

d. Analisis Output

Program berhenti setelah kedua thread mencoba mengunci mutex kedua.

5. Analisis Hasilnya

- a. Apakah program berjalan sesuai ekspektasi?
- b. Ya, program menunjukkan deadlock seperti yang diharapkan.
- c. Masalah yang Ditemui
 - Program tidak pernah selesai.
 - Menggunakan `sleep(1)` untuk memaksa timing tertentu dan memperbesar peluang deadlock.
- d. Performa Program
 - CPU usage rendah (program hanya menunggu).
 - Thread tidak melakukan pekerjaan apapun setelah deadlock.
- e. Pelajaran yang Didapat
 - Deadlock bisa terjadi jika empat kondisi Coffman terpenuhi.
 - Urutan penguncian resource sangat penting.
 - Penting memahami mekanisme sinkronisasi dalam program multithreading.

B. Tugas 2 — Implementasi Pencegahan Deadlock

➤ Mencoba metode lock ordering

1. Metode Lock Ordering

Ide:

Semua thread harus mengunci mutex dalam urutan yang sama, misalnya:

Selalu kunci A dulu → baru kunci B

Kalau semua thread mengikuti urutan ini, circular wait tidak akan terjadi.

2. Kode: deadlock_lock_ordering.c

nano deadlock_lock_ordering.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA;
pthread_mutex_t lockB;

// Fungsi helper untuk lock dengan urutan yang benar
void lock_in_order(pthread_mutex_t* first, pthread_mutex_t* second) {
    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
}

void unlock_in_order(pthread_mutex_t* first, pthread_mutex_t* second) {
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}

void* thread1(void* arg) {
    printf("T1: mencoba lock A lalu B (urutan benar)\n");
    lock_in_order(&lockA, &lockB);

    printf("T1: berhasil lock A dan B\n");
    sleep(1);

    unlock_in_order(&lockA, &lockB);
    printf("T1: unlock selesai\n");

    return NULL;
}

void* thread2(void* arg) {
```

```

printf("T2: mencoba lock A lalu B (urutan benar)\n");
lock_in_order(&lockA, &lockB);

printf("T2: berhasil lock A dan B\n");
sleep(1);

unlock_in_order(&lockA, &lockB);
printf("T2: unlock selesai\n");

return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

Jalankan
gcc deadlock_lock_ordering.c -o
./ deadlock_lock_ordering

3. Output

```

T1: lock A berhasil
T1: lock B berhasil
T1: selesai
T2: lock B berhasil
T2: lock A berhasil
T2: selesai

```

Disini tidak ada deadlock, Karena urutan penguncian sama untuk semua thread.

➤ Mencoba Metode Try-Lock dengan Retry

1. Metode Try-Lock dengan Retry

Ide:

Gunakan pthread_mutex_trylock()

- Jika gagal lock B, thread melepaskan lock A

- Lalu coba lagi dari awal
- Tidak ada kondisi hold & wait -- deadlock terhindari

2. Kode Program

nano deadlock_trylock_retry.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA;
pthread_mutex_t lockB;

void* thread1(void* arg) {
    while (1) {
        pthread_mutex_lock(&lockA);
        printf("T1: lock A berhasil\n");

        if (pthread_mutex_trylock(&lockB) == 0) {
            printf("T1: lock B berhasil\n");

            // Critical section
            sleep(1);

            pthread_mutex_unlock(&lockB);
            pthread_mutex_unlock(&lockA);
            printf("T1: selesai\n");
            break;
        } else {
            printf("T1: gagal lock B → retry\n");
            pthread_mutex_unlock(&lockA);
            usleep(100000); // 0.1 detik
        }
    }
    return NULL;
}
```

```

}

void* thread2(void* arg) {
    while (1) {
        pthread_mutex_lock(&lockB);
        printf("T2: lock B berhasil\n");

        if (pthread_mutex_trylock(&lockA) == 0) {
            printf("T2: lock A berhasil\n");

            sleep(1);

            pthread_mutex_unlock(&lockA);
            pthread_mutex_unlock(&lockB);
            printf("T2: selesai\n");
            break;
        } else {
            printf("T2: gagal lock A → retry\n");
            pthread_mutex_unlock(&lockB);
            usleep(100000);
        }
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

```

```

pthread_join(t1, NULL);
pthread_join(t2, NULL);

return 0;
}

```

3. Output

```

T1: lock A berhasil
T1: lock B berhasil
T1: selesai
T2: lock B berhasil
T2: lock A berhasil
T2: selesai

```

Berhasil juga tidak terjadi deadlock, Karena:

- Saat gagal lock resource kedua
- Thread melepaskan lock pertama
- Lalu mencoba lagi
- Tidak ada circular wait

4. Perbandingan Ke-Duanya

Aspek	Lock Ordering	Try-lock + Retry
Cara kerja	Semua thread lock resource dalam urutan yang sama	Jika gagal lock resource kedua → retry
Kelebihan	Sederhana, efisien, deterministik	Sangat fleksibel, cocok untuk banyak resource
Kekurangan	Harus tahu urutan resource sejak awal	Bisa menyebabkan spin-loop (busy wait)
Kinerja	Lebih cepat (tidak retry)	Sedikit lebih lambat
Skenario ideal	Sistem dengan urutan resource tetap	Sistem dinamis, resource banyak & kompleks
Mencegah deadlock?	Ya, 100% jika urutan dipatuhi	Ya, karena tidak hold & wait

C. Tugas 3: Program Banker's Algorithm

1. Analisis Masalah

Pada tugas ini, diminta membuat program Banker's Algorithm dengan:

- Jumlah proses: 4 proses (P0, P1, P2, P3)
- Resource: 3 jenis (R0, R1, R2)
- Available awal: [5, 4, 3]

Program harus:

- Menghitung matriks Need
- Memproses permintaan resource dari proses tertentu
- Mengecek apakah sistem berada dalam safe state
- Menolak atau menerima request berdasarkan hasil pengecekan keamanan

2. Identifikasi Masalah

- Sistem harus menghindari deadlock ketika proses meminta resource.
- Jika resource diberikan tanpa pengecekan, sistem bisa masuk unsafe state.
- Bankers Algorithm membantu mencegah deadlock dengan memastikan:
- Resource tidak diberikan jika mengakibatkan sistem tidak aman.

3. Disain Solusi

Solusi dirancang dengan tahapan:

1. Hitung matriks Need:

$$\text{Need} = \text{Max} - \text{Allocation}$$

2. Setiap permintaan:

- Periksa apakah $\text{request} \leq \text{need}$
- Periksa apakah $\text{request} \leq \text{available}$
- Lakukan trial allocation
- Jalankan Safe State Check
- Jika aman request disetujui
Jika tidak rollback

3. Simulasi 2 request dari dua proses berbeda.

4. Penjelasan Algoritma

Banker's Algorithm terdiri dari dua bagian utama:

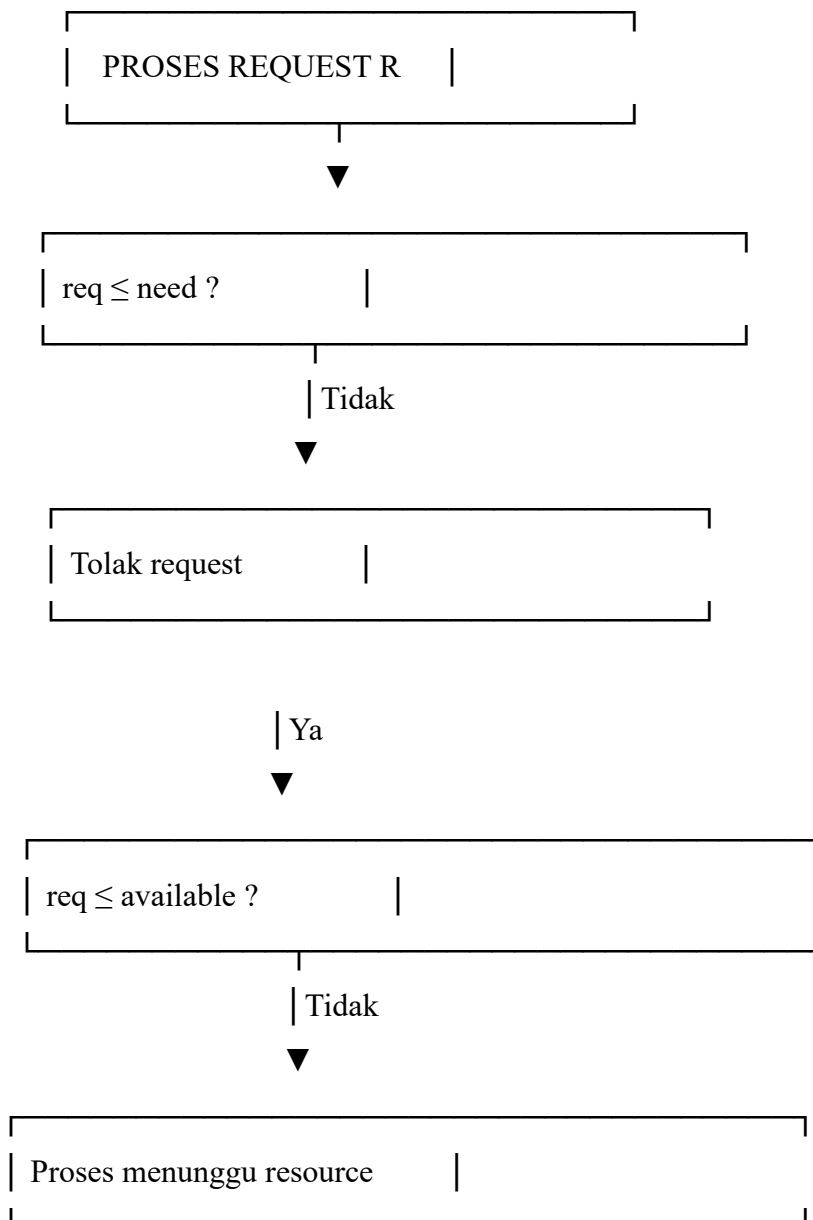
a. Resource Request Algorithm

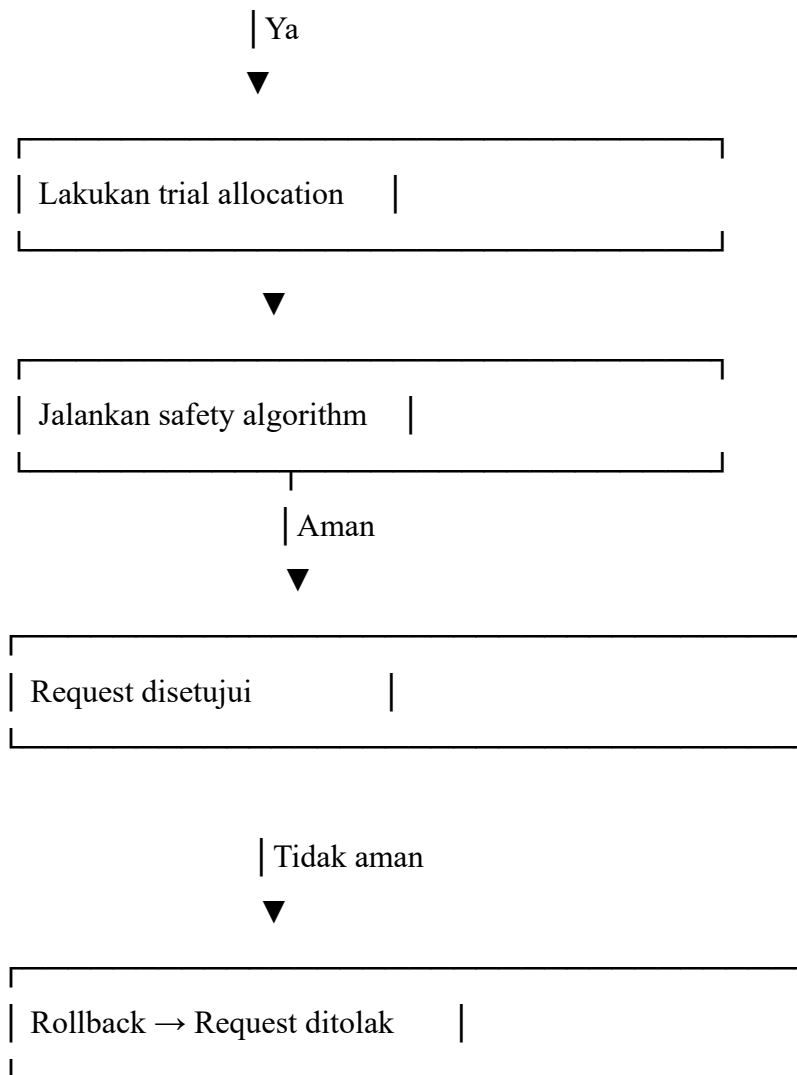
- Digunakan ketika suatu proses meminta resource
- Mengecek apakah permintaan dapat dipenuhi tanpa membuat sistem tidak aman

b. Safety Algorithm

- Mengecek apakah ada safe sequence
- Jika semua proses bisa mendapatkan resource dan menyelesaikan eksekusi safe state
- Jika tidak unsafe

5. Flowchart





6. Kode Program

nano bangkers.c

```

Compile dan Jalankan
gcc bangkers.c -o bangkers
./bangkers

```

7. Output

```

Request dari P1: [1 0 2]
Sistem AMAN. Request DISETUJUI.

Request dari P0: [3 3 0]
Sistem AMAN. Request DISETUJUI.

```

- Test Case 1 – Request P1 = [1,0,2]

Terminal Output:

Request dari P1: [1 0 2]

Sistem aman. Request disetujui..

Safe state ditemukan → permintaan DITERIMA.

- Test Case 2 – Request P0 = [3,3,0]

Output:

Request dari P0: [3 3 0]

Sistem AMAN ! Request DISETUJUI

8. Penjelasan Data

- Available

{5, 4, 3}

- Maximum

P	R0	R1	R2
P0	3	2	2
P1	6	1	3
P2	3	1	4
P3	4	2	2

- Allocation

P	R0	R1	R2
P0	1	0	0
P1	5	1	1
P2	2	1	1
P3	0	0	2

Need = Maximum – Allocation

Perhitungan:

- P0

Max = (3, 2, 2)

Alloc = (1, 0, 0)

Need = (3-1, 2-0, 2-0)

Need P0 = {2, 2, 2}

- P1

Max = (6, 1, 3)

Alloc = (5, 1, 1)

Need = (6-5, 1-1, 3-1)

Need P1 = {1, 0, 2}

- P2

Max = (3, 1, 4)

Alloc = (2, 1, 1)

Need = (3-2, 1-1, 4-1)

Need P2 = {1, 0, 3}

- P3

Max = (4, 2, 2)

Alloc = (0, 0, 2)

Need = (4-0, 2-0, 2-2)

Need P3 = {4, 2, 0}

9. Analisis Hasil

a. Apakah program berjalan sesuai ekspektasi?

Ya. Program berhasil:

- Mengidentifikasi permintaan yang aman
- Menolak request yang menyebabkan unsafe state
- Menunjukkan safe/unsafe state sesuai Banker's Algorithm

b. Masalah yang Ditemui & Solusinya

Masalah	Penyebab	Solusi
Hitung Need salah	Max/Allocation tidak sesuai	Periksa data matriks
Sistem selalu unsafe	Trial tidak dikembalikan	Tambahkan rollback
Output tidak muncul	Tidak kompilasi pakai -g	Perbaiki perintah compile

c. Performa Program

- Cepat karena hanya menggunakan operasi matriks sederhana
- Kompleksitas $O(P^2 \times R)$
- Tidak menggunakan thread → tidak ada overhead sinkronisasi

d. Pelajaran yang Didapat

- Banker's Algorithm efektif mencegah deadlock sebelum terjadi
- Pentingnya safe state dalam alokasi resource
- Trial allocation wajib dilakukan sebelum mengubah resource secara permanen

D. TUGAS 4: Kasus Nyata - Simulasi Deadlock Transfer Uang

1. Soal Pemrograman

a. Analisis Masalah

Pemahaman Soal

Kasus: Dua orang melakukan transfer uang secara bersamaan.

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1

Jika kedua proses mencoba mengunci rekening secara terbalik, maka dapat terjadi deadlock.

b. Identifikasi Masalah Sinkronisasi

Deadlock terjadi karena:

- Thread A mengunci Rekening 1, Thread B mengunci Rekening 2.
- Keduanya menunggu lock satu sama lain → kondisi saling menunggu.

c. Desain Solusi (Deadlock Prevention)

Metode pencegahan deadlock:

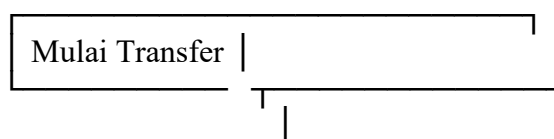
- Menggunakan pengurutan global lock ordering (misal: selalu mengunci rekening dengan ID terkecil dulu). Dengan cara ini, tidak ada dua thread yang mengambil lock dalam urutan berbeda.

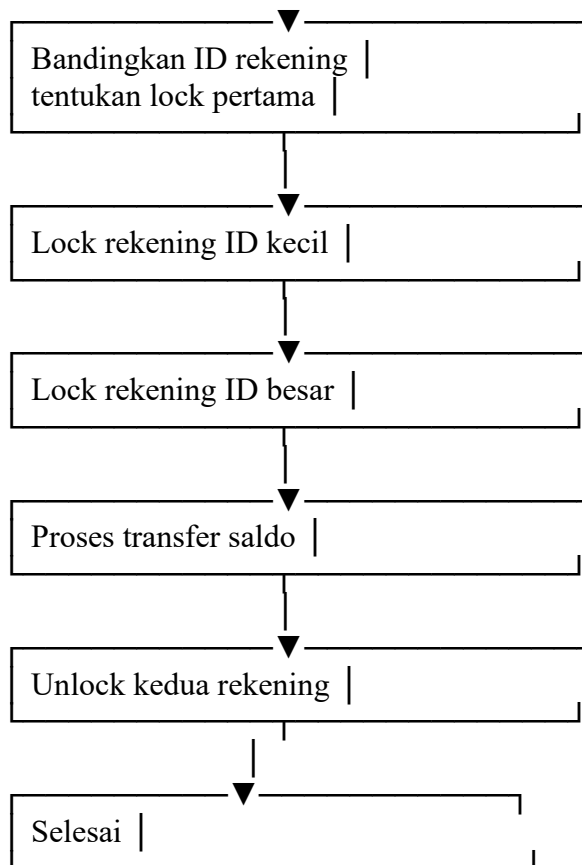
2. Implementasi

a. Penjelasan Algoritma

- Setiap rekening memiliki objek lock.
- Untuk melakukan transfer, program membandingkan ID rekening.
- Lock diambil berdasarkan urutan terkecil ke terbesar.
- Transfer dilakukan aman tanpa deadlock.

b. Flowchart





c. Kode deadlock.c – sengaja bikin deadlock

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int saldo1 = 1000;
int saldo2 = 1000;

pthread_mutex_t lock1;
pthread_mutex_t lock2;

// Orang A: Rekening 1 -> Rekening 2
void* transferA(void* arg) {
    pthread_mutex_lock(&lock1);
    printf("A mengunci Rek 1\n");
```

```

sleep(1); // biar kemungkinan deadlock makin besar

pthread_mutex_lock(&lock2);
printf("A mengunci Rek 2\n");

saldo1 -= 100;
saldo2 += 100;

pthread_mutex_unlock(&lock2);
pthread_mutex_unlock(&lock1);

return NULL;
}

// Orang B: Rekening 2 -> Rekening 1
void* transferB(void* arg) {
    pthread_mutex_lock(&lock2);
    printf("B mengunci Rek 2\n");
    sleep(1);

    pthread_mutex_lock(&lock1);
    printf("B mengunci Rek 1\n");

    saldo2 -= 100;
    saldo1 += 100;

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {

```

```

pthread_t a, b;

pthread_mutex_init(&lock1, NULL);
pthread_mutex_init(&lock2, NULL);

printf("\n=== SIMULASI DEADLOCK ===\n");

pthread_create(&a, NULL, transferA, NULL);
pthread_create(&b, NULL, transferB, NULL);

pthread_join(a, NULL);
pthread_join(b, NULL);

printf("\nSaldo Akhir Rek 1: %d\n", saldo1);
printf("Saldo Akhir Rek 2: %d\n", saldo2);

return 0;
}

```

```

compile dan jalankan
gcc deadlock.c -o deadlock -lpthread
./deadlock.

```

d. Output

```

=== SIMULASI DEADLOCK ===
A mengunci Rek 1
B mengunci Rek 2

```

Program akan macet / hang tanda deadlock terjadi.

e. Kode Deadlock Prevention (mengatasi masalah)

Teknik: Global Lock Ordering

- Selalu kunci rekening dalam urutan ID kecil ke ID besar
- Thread tidak mungkin saling tunggu.

Buat File Program nano prevention.c

```

#include <stdio.h>
#include <pthread.h>

```

```

#include <unistd.h>

int s1 = 1000;
int s2 = 1000;

pthread_mutex_t l1;
pthread_mutex_t l2;

// selalu kunci l1 dulu baru l2
void lock_ordered() {
    pthread_mutex_lock(&l1);
    pthread_mutex_lock(&l2);
}

void unlock_ordered() {
    pthread_mutex_unlock(&l2);
    pthread_mutex_unlock(&l1);
}

void* amanA(void* arg) {
    lock_ordered();
    printf("A melakukan transfer dengan aman\n");
    s1 -= 100;
    s2 += 100;
    unlock_ordered();
    return NULL;
}

void* amanB(void* arg) {
    lock_ordered();
    printf("B melakukan transfer dengan aman\n");
    s2 -= 100;
    s1 += 100;
}

```

```

unlock_ordered();
return NULL;
}

int main() {
    pthread_t a, b;

    pthread_mutex_init(&l1, NULL);
    pthread_mutex_init(&l2, NULL);

    printf("\n=== DEADLOCK PREVENTION ===\n");

    pthread_create(&a, NULL, amanA, NULL);
    pthread_create(&b, NULL, amanB, NULL);

    pthread_join(a, NULL);
    pthread_join(b, NULL);

    printf("\nSaldo Akhir Rek 1: %d\n", s1);
    printf("Saldo Akhir Rek 2: %d\n", s2);

    return 0;
}

```

Compile dan jalankan

```
gcc prevention.c -o prevention -lpthread
```

./prevention

f. Output

```

=== DEADLOCK PREVENTION ===
A melakukan transfer dengan aman
B melakukan transfer dengan aman

Saldo Akhir Rek 1: 1000
Saldo Akhir Rek 2: 1000

```

Deadlock terjadi karena 4 kondisi simultan (Coffman Conditions):

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait terjadi saat A menunggu B, B menunggu A

[Deadlock Prevention]

Dengan menetapkan aturan:

Selalu kunci Rek 1 dulu, baru Rek 2.

Maka circular wait tidak mungkin terjadi.

KESIMPULAN

Secara keseluruhan, praktikum ini menegaskan bahwa:

- Deadlock adalah masalah fundamental dalam sistem operasi dan pemrograman multithread.
- Penyebab deadlock dapat dijelaskan menggunakan kondisi Coffman. Deadlock dapat dicegah dengan strategi sederhana seperti lock ordering, atau teknik adaptif seperti try-lock.
- Deadlock dapat dihindari secara matematis dengan Banker's Algorithm.
- Studi kasus nyata menunjukkan bahwa deadlock memiliki dampak serius bila tidak dicegah.

Pemahaman ini penting agar program multithread yang dibuat lebih aman, stabil, dan bebas dari deadlock.